挑战点8：硬件子系统接收并解析遥控器命令

R8EF 接收机基本介绍

> 来自乐迪官网

适用发射机型号

Radiolink R8EF 2.4G 八通道接收机，适用于乐迪八通道遥控器T8FB、T8S、RC6GS V2、RC4GS V2，支持 S-BUS、PPM 和 PWM 三种信号同时输出。

信号输出类型

支持 SBUS、PPM 和 PWM 三种信号同时输出。

信号工作模式

1. PWM 信号工作模式

接收机指示灯为红色，输出8个通道的 PWM 信号。

2. S-BUS/PPM 信号工作模式

接收机指示灯为蓝色（紫灯），1通道输出 S-BUS 信号，2通道输出 PPM 信号，3-8通道输出对应的 PWM 信号，共计输出8个通道的信号。

信号切换

短按接收机侧面的（ID SET）开关两次（一秒内），完成 R8EF 的 PWM 信号和 SBUS 或者 PPM 信号的切换。

对码

每个发射机都有独立的 ID 编码。开始使用设备前，接收机必须与发射机对码。

对码完成后，ID 编码则储存在接收机内，且不需要再次对码。当您购买了新的 R8EF 接收机，必须要重新对码，否则接收机将无法正常使用。

1. 将发射机和接收机放在一起，两者距离50厘米左右。
2. 打开发射机电源开关使遥控器处于开机状态，再给接收机上电，R8EF 接收机将寻找与之最近的的遥控器进行对码。
3. 按下接收机侧面的（ID SET）开关1秒以上直至接收机指示灯开始闪烁便松开，当接收机指示灯停止闪烁变为常亮时，表示对码完成。
4. 如果对码操作后，接收指示灯仍然慢闪提示，表示对码失败，请重复上述操作。

温馨提示：在发射机和接收机连接电源的情况下，当接收机和发射机未对码成功时或者飞行时接收机丢失信号时，接收机的指示灯会慢闪提示。

接收机的连接

接收机的连接使用杜邦线，常见为白/红/黑色线或者黄/红/棕色线。两种舵机线都是浅色线为信号线，深色线为地线，中间为5V供电，三根线分别与接收机标签“ ⊓+- ”相对应，连接舵机线时，信号线（浅色线）向上，地线朝下插入接收机对应的通道。

注意：乐迪接收机都设计有电子防反插功能，当舵机线插反时，接收机不工作，且不会因为杜邦线接反损坏。接收机所有通道的地线均并联在一起，接收机供电电压为4.8V - 10V，务必注意当接收机使用单独的电池独立供电时，电池接反接收机不会损坏，但此时连接舵机，会导致舵机损坏。

接收机天线安装

1. 尽量保证天线笔直，否则将会减小有效控制范围。
2. 大型的模型机可能会存在影响信号发射的金属部件，在这种情况下，天线应处于模型的两侧。这样在任何飞行姿态下都能保持拥有最佳的信号状态。
3. 天线应该尽可能远离金属导体和碳纤维，至少要有半英寸的距离，但不能过度弯曲。
4. 尽可能保持天线远离马达、电子调速器（ESC）和其他可能的干扰源。在实际安装接收机的过程中。
5. 可以使用海绵或者是泡沫材料将其绕起来用以防震。
6. 接收机包含一些高精度的电子零部件。因此在使用时，请小心轻放，防止剧烈震动或处于高温环境中。
7. 为了更好地保护接收机，用 R/C 专用泡沫或橡胶布等防震材料将其缠绕。为了防止接收机受潮，最好是将其放到塑料袋中并把袋口封好。如果有水分进入接收机，可能造成间歇性失控甚至完全失去控制。将接收机放入塑料袋还可以防止燃料以及残渣进入机身。

警告：连接程序完成以后，请将发射机关机再开机，则程序生效，以确认接收机确实与发射机连接正常并受发射机的控制。当电动机的动力电线连接于调速器时，或在发动机工作时不要执行连接程序，这时操作可能会导致严重伤害。

技术参数

通道数：8通道
工作电压：3-15V
工作电流：30mA（输入电压5V）
尺寸：长 * 宽 * 高=48.5 * 21 * 11mm
重量：7g
分辨率：4096，每级0.5us高精度，让所有舵机静若止水
距离：空中2000米左右，实际操控距离和飞行环境有关

方案选择

我们的方案是 SBUS + 反向器 + DMA + IDLE 中断。

为什么要用 SBUS？

核心原因是**用极少的硬件资源，可靠地拿到足够多的控制量。**

- **PWM**：一个通道一根线。四轴至少需要 4 个通道（油门、俯仰、横滚、偏航），如果想加模式切换、解锁等功能，8 通道就要占用 8 个 MCU 引脚，而且每根线都需要定时器输入捕获，IO 和定时器资源消耗都很大。

- **PPM**：单线传输多通道，但本质是**模拟脉宽时序。**STM32 需要定时器捕获每个脉冲的宽度，通道数受限（R8EF 的 PPM 最多 8 通道），抗干扰性也一般。
- **SBUS**：单根信号线，**原生 UART 串口帧。**R8EF 输出 SBUS 时，数据以 100000 bps 的串口帧格式到达 STM32，不需要定时器捕获，只需要一个 USART 接收引脚。协议层面可承载 **16 个比例通道 + 2 个数字开关通道**，对于四轴控制绰绰有余。

> **SBUS 把“时序测量问题”变成了“串口字节处理问题”。**我们不需要关心脉冲有多宽，只需要在内存里按位拆分字节即可。
>
> R8EF 本身是 8 通道接收机，在 SBUS 模式下，**只有第 1 通道物理引脚输出 SBUS 信号**，第 2 通道输出 PPM，第 3–8 通道输出 PWM。但 SBUS **协议帧**里仍然按 16 通道的格式打包，实际有效的通道数取决于遥控器端映射了几个通道。你的解析代码可以按 16 通道统一提取，后续电机驱动模块只取用实际有效的部分即可。

为什么是 DMA？

对比维度	中断逐字节接收	DMA + IDLE 中断
中断频率	每字节一次，约 1786 次/秒	每帧一次（IDLE 触发），约 71 次/秒
CPU 占用	极低（~0.2%–0.4%），但每次中断都有压栈/出栈开销	几乎为零，CPU 只在帧结束时介入一次
数据搬运	CPU 亲自执行 LDR/STR 逐字节搬运	DMA 控制器后台自动搬运，不占用 CPU 指令周期
实现复杂度	简单直接，与你们已有的汇编中断接收方案完全吻合	需要额外配置 DMA 通道、循环模式、IDLE 中断，初始化更复杂
教学展示价值	能完整展示寄存器级中断处理、汇编 ISR、缓冲区管理	会抽象掉“逐字节搬运”这一层，不利于展示底层细节
适用场景	低速率、帧短、主循环负担不极端的场景	高速率、大帧长、主循环计算密集的场景

理论上 DMA+IDLE 能让 CPU 更“干净”，DMA + IDLE 是性能更优的工程替代方案。

接收机工作原理

SBUS 接收机是周期性、不间断地向外发送完整帧的，无论摇杆是否动作。

这和“蓝牙式按需发送”完全不同。SBUS 的设计逻辑是：**接收机持续把当前所有通道的最新值打包成帧，以固定间隔循环发送。**摇杆不动时，发送的通道值保持不变；摇杆移动时，下一帧中的数据自然更新。

发送频率取决于 SBUS 的工作模式：

模式	帧间隔	大约帧率
普通模式	约 **14 ms** 一帧	约 71 Hz
高速模式	约 **7 ms** 一帧	约 143 Hz

也就是说，即使手远离遥控器，接收机仍然在**每 14 ms（或 7 ms）** 雷打不动地发出一帧 25 字节的数据。**帧与帧之间没有“静默期”之说**——一帧发完约 2.5 ms 后，间隔一小段空闲，下一帧就又来了，周而复始。

SBUS 通信

SBUS 在物理层和链路层，本质上就是一个“反向（Inverted）的串口通信”。

除了电平极性是反的，它的数据格式和时序完全遵循标准的 UART（异步串行通信）规范。这意味着你不需要像解码红外遥控那样去算脉宽，只需要把它当成一个特殊的串口来读就行了。

它作为串口，标准的参数配置如下（业界俗称 **100k, 8E2**）：

参数名	参数值	具体含义
波特率 (Baud Rate)	**100,000 bps**	即 100k。这意味着每一位数据的持续时间是 10μs。
数据位 (Data Bits)	**8 位**	标准的一个字节长度。
校验位 (Parity)	**偶校验 (Even)**	保证数据帧中“1”的总数为偶数，用于检错。
停止位 (Stop Bits)	**2 位**	一个字节发完后，保持 2 个位时间的高电平（反向电平下为低电平），用于帧同步。

为什么这些参数要这样设置?

1. **100k 波特率**：这是 Futaba 制定 SBUS 标准时定下的死规定。相较于常见的 9600 或 115200，100k 是一个特定的速率。**在你的代码中，要注意 BRR 寄存器的计算。**（例如在 16MHz 时钟下，BRR 的值正好是 0x0A，你队友代码里也是这么写的）。
2. **8 位数据位**：正好对应一个字节，方便按字节打包。
3. **偶校验 + 2 个停止位**：
    - 遥控器是模型飞机的命脉。在空中，电磁环境复杂，为了防止一个比特翻转导致飞机炸机，SBUS 增加了**偶校验**来检测单比特错误（如果校验失败，STM32 的 USART 会产生 PE 错误标志）。
    - **2 个停止位**不是为了防错，而是为了给接收机内部或单片机留出更多的处理时间余量，也增加了帧与帧之间的抗干扰间隔。

SBUS 协议

SBUS 是完全标准化的协议，帧头、帧尾、载荷排布都是固定的，直接按规范解析即可，不需要像蓝牙通信那样自己制定包头包尾。

SBUS 每帧固定 **25 字节**，结构如下：

字节位置	内容
Byte 1	帧头，**固定** `0x0F`
Byte 2–23	22 字节通道数据（16 个 11 位通道，共 176 bit）
Byte 24	标志位字节（含帧丢失、Failsafe 等状态）

字节位置	内容
Byte 25	帧尾，**固定** `0x00`

通道数据的位排布也是标准化的：16 个通道各占 **11 bit**，**低位在前、高位在后**连续跨字节打包，取值范围 **0–2047**。
经典的首通道提取公式是：

```
CH1 = buffer[1] + ((buffer[2] & 0x07) << 8)
```

之前蓝牙“自己制定包”，在 SBUS 这里**完全不需要**——你只需要**按照既定格式做模式匹配和位拼接。**状态机的作用是在字节流中找到 `0x0F` 帧头，然后连续收满 25 字节并校验 `0x00` 帧尾，而不是发明一套新的包结构。

> SBUS 帧尾在绝大多数文档和 R8EF 输出中是 `0x00`，少数资料提到 `0x7E` 的情况属于特定实现变体。你们以实测 R8EF 输出为准，黑板板书按 `0x00` 讲解即可。

SBUS 协议内容

22字节通道数据（Channel Data）

这22个字节里，**装的是16个遥控通道的“位置”信息**，比如摇杆的位置、旋钮的角度。

- **里面装的是什么数据?**
  每个通道的值用一个 **11位（11-bit）** 的整数表示，所以它的数值范围是 **0 到 2047**（2^{11} = 2048）。
  - **物理含义**：这个数值代表了遥控器上对应通道的“杆量”或“开关位置”。
  - **与 PWM 的映射**：在工程应用中，常常会把这个0-2047的范围**映射到标准的 PWM 脉宽范围**（通常是1000μs到2000μs），方便与传统的舵机或电调兼容。例如，一个常见的映射关系是：SBUS 值 **192–1792** 对应 PWM 值 **1000–2000μs**。数值 **1024** 通常代表摇杆的**中位**。
- **怎么解析？（核心：位打包）**
  这22个字节（共176位）是**无缝打包**的，每个通道的11位数据**不是按字节对齐的**，而是“你挤着我，我挨着你”地连续存放。解析时，需要像拼图一样，从连续的字节中**按位（bit）** 提取。

  标准的打包顺序是 **LSB First（最低有效位优先）**，即一个通道的低位先出现，高位后出现。

  解析示例：
  - **CH1**：由 `buffer[1]` 的全部8位（低8位）和 `buffer[2]` 的低3位（高3位）组成。
    伪代码：`CH1 = buffer[1] | ((buffer[2] & 0x07) << 8)`
  - **CH2**：由 `buffer[2]` 的高5位（低5位）和 `buffer[3]` 的低6位（高6位）组成。
    伪代码：`CH2 = (buffer[2] >> 3) | ((buffer[3] & 0x3F) << 5)`
  - **以此类推...**

  解析需要做的就是写一个循环或展开的汇编代码，对这22个字节进行连续的**移位（Shift）、与（AND）、或（OR）** 操作，最终还原出16个0-2047的整数。

1字节标志位 (Flag Byte)

这1个字节（Byte 24）包含了**数据有效性**和**额外数字通道**的状态信息。它的位定义如下：

Bit位	十六进制掩码	含义	说明
Bit 7	`0x80`	**数字通道 CH17**	开关量，例如用于解锁/上锁
Bit 6	`0x40`	**数字通道 CH18**	另一个开关量
Bit 5	`0x20`	**Frame Lost（帧丢失）**	置1表示**上一帧数据丢失**，接收机与发射机之间出现了丢包
Bit 4	`0x10`	**Failsafe（失控保护）**	置1表示**接收机已进入失控保护模式。**通常是在连续丢失多帧后触发
Bit 3-0	-	**保留 (n/a)**	目前未使用

怎么解析?
在状态机中，当一帧25字节数据被成功接收并校验后，只需检查 `buffer[23]` 这个标志字节的对应位即可。

- 检查 `buffer[23] & 0x20` 是否非零，以判断是否丢帧。
- 检查 `buffer[23] & 0x10` 是否非零，以判断是否触发了失控保护。
- 检查 `buffer[23] & 0x80` 和 `0x40`，可以获取两个额外的数字开关通道状态。

包头（0x0F）和包尾（0x00）的特殊含义与安全性

它们本质上是“帧定界符”。包头 `0x0F` 和包尾 `0x00` **并不包含像校验和（Checksum）或CRC那样的、用于验证数据正确性的“特殊含义”。**它们的作用非常纯粹：**告诉解析程序“一帧数据从这里开始，到这里结束”**，是数据流中的**同步标志（Sync Marker）。**

绝对不能保证中间数据不出现包头/包尾。 在 SBUS 的22个通道数据字节中，**完全有可能出现数值恰好等于 `0x0F` 或 `0x00` 的字节。**

但这通常**不会导致解析错误**，原因在于SBUS解析依赖的是**严格的“位置”和“上下文”**，而非简单的“搜索关键字”：

1. **固定长度是根本保障**：SBUS 协议规定，**一帧必须是固定25个字节。**你的状态机在找到 `0x0F` 后，会**强制性地、连续地接收后续的24个字节**，它不会去检查中间字节的值，而是无条件地按位置将它们视为数据、标志位或帧尾。
2. **状态机的上下文**：你的解析逻辑是在**“等待帧头”**的状态下才去寻找 `0x0F`。一旦帧头被确认，状态机就切换到**“接收数据”**状态，此时即使收到 `0x0F`，它也会被当作**普通数据**存入缓冲区，而不会被误认为是新的帧头。

SBUS 存在**数据流错位**的风险。例如，如果因为干扰**丢失了一个字节**，那么你的状态机可能会把**通道数据中的某个 `0x0F` 误认为是帧头**，从而开始接收一个“错误的帧”。

解决方案：这就是我们之前讨论的**帧级校验和重同步机制**的用武之地。

1. **完整性校验**：当状态机收满25字节后，进行双重检查：`buffer[0] == 0x0F` 且 `buffer[24] == 0x00`。
2. **滑动窗口重同步**：如果校验失败，说明发生了错位。此时，不能简单地将整个缓冲区丢弃，而应该**将缓冲区的起始位置向后“滑动”一个字节**（即丢弃 `buffer[0]`），然后**重新从 `buffer[1]` 开始寻找下一个 `0x0F`**，以此类推，直到重新找到正确的帧头并收满一个合法的25字节帧。

DMA 工作原理

DMA 控制器是一个独立于 CPU 的总线主设备。一旦被配置好，它就可以在后台自动搬运数据，完全不需要 CPU 的干预。

基本概念

概念	说明
DMA 控制器 (Controller)	STM32F4通常有 **DMA1** 和 **DMA2** 两个控制器，其中 DMA2 仅在大容量产品中存在。
数据流 (Stream)	每个 DMA 控制器有 **8 个独立的数据流**，可以理解为8条并行的“传输通道”。
通道 (Channel)	每个数据流可以选择 **8 个通道** 中的一个。通道决定了这个数据流服务于哪个外设的请求（如 USART2_RX、ADC 等）。
仲裁器 (Arbiter)	当多个数据流同时发出请求时，由仲裁器根据**软件优先级**（非常高/高/中/低）和**硬件优先级**（流编号越小优先级越高）来决定谁先传输。
FIFO	每个数据流都有一个4字深的 FIFO，用于在源和目标之间临时缓存数据，可以设置阈值（1/4、1/2、3/4、满）来平衡传输速率。

关键映射关系：对于 USART2 的接收（USART2_RX），在 STM32F4 上，标准的硬件映射是 **DMA1 的 Stream 5，Channel 4。**

DMA 工作核心流程

DMA 的传输由一系列寄存器控制，核心工作流程可以概括为“配置-触发-传输-计数”：

1. **配置传输属性**：通过 `DMA_SxCR` 寄存器设定传输方向（外设→内存）、数据宽度（字节）、**内存地址递增**（`MINC`，让数据依次填入缓冲区）、**外设地址不递增**（`PINC`，始终从 USART_DR 读取），以及**循环模式**（`CIRC`）。循环模式意味着传输完设定的数据量后，地址和计数器自动重置，无需 CPU 干预即可开始下一轮搬运。
2. **设定地址与数据量**：通过 `DMA_SxPAR` 设定外设地址（`&USART2->DR`），`DMA_SxM0AR` 设定内存缓冲区地址，`DMA_SxNDTR` 设定本次要传输的数据总个数。
3. **使能与触发**：使能数据流后，DMA 进入待命状态。每当 USART2 收到一个字节，其 `RXNE` 标志置位，会向 DMA 发出一个请求（前提是 USART 的 `CR3` 寄存器中使能了 `DMAR` 位）。DMA 响应请求，将 `USART2->DR` 中的1字节搬运到内存，然后 `NDTR` **计数器减1。**
4. **完成与循环**：当 `NDTR` 减到0时，如果配置为循环模式，DMA 会自动将 `NDTR` 重载为初始值，并将内存地址指针重置到缓冲区起始位置，开始下一轮搬运。

IDLE（空闲中断）

在 STM32 的 USART 外设中，**IDLE 代表“总线空闲状态”。**
串口通信在没有数据传输时，信号线默认保持高电平。当接收端检测到**高电平持续时间超过了一个完整的字节传输时间**（对于 100kbps 的 8E2 格式，一个字节传输大约需要 120μs），硬件就会认为“当前数据发完了，总线上没人说话了”，此时会自动把 `USART_SR` 寄存器里的 `IDLE` 标志位置 1。如果开启了 IDLE 中断，CPU 就会立刻进入中断服务函数。

SBUS 接收机发送一帧 25 个字节，这 25 个字节是**紧密相连、中间毫无停顿**地发出来的。传输这 25 个字节大约需要 **2.5ms**（如果算上 8E2 格式的开销，约需要 3ms）。

- **在这 3ms 期间**：总线上一直有数据在翻转，**绝对不可能触发 IDLE 。**
- **在第 25 个字节发完的瞬间**：最后一个停止位结束，总线恢复高电平。只要总线保持高电平超过 **1 个字节的时间（约 120μs）**，**IDLE 中断就会立刻触发。**

所以，**IDLE 中断不是等那几毫秒（14ms 或 7ms）的间隔快结束时才触发，而是在这一波数据刚刚发完的第 120μs 就触发了。** 触发完之后，串口保持空闲，直到下一个 14ms 周期，下一帧数据到来。

如果发生了"丢包"或"错位"，IDLE 中断怎么表现?

因为 IDLE 是基于**时间（总线空闲超时）**触发的，它非常聪明：

- 如果接收机发到第 10 个字节时，因为干扰，后面的数据全丢了。那么总线在第 10 个字节发完后，就会保持高电平。
- 过了 120μs，**IDLE 中断依然会触发。**
- 此时你读 DMA 的 `NDTR`，发现只收到了 10 个字节（而不是标准的 25 个）。你立刻就能断定：**这一波数据不完整，直接丢弃，或者把它喂给状态机去滑动找帧头。**

SBUS + DMA + IDLE 接收流程

将 DMA 引入后，SBUS 的接收流程在**中断层面被极大简化**，但**主循环的解析逻辑依然需要状态机。**核心的 " 黄金组合 " 是：**DMA 负责搬运，IDLE 中断负责通知 " 一帧结束 " 。**

硬件层：DMA + IDLE 自动搬运

1. **初始化**：配置 DMA1_Stream5/Channel4 为**外设→内存、循环模式**，内存地址递增，数据宽度为字节。将 `NDTR` 设置为一个足够大的值（例如等于你的缓冲区大小，如512）。同时，使能 USART2 的 DMA 接收请求（`CR3` 的 `DMAR` 位）。
2. **自动接收**：R8EF 连续发送 SBUS 数据。每来一个字节，USART2 的 `RXNE` 触发 DMA 请求，DMA 自动将字节从 `DR` 寄存器搬运到指定的 `rx_buf` 数组中，`NDTR` 递减，**全程无需 CPU 参与。**
3. **IDLE 中断触发**：一帧25字节发送完毕后，接收线上会出现至少一个字节时间的空闲。USART2 的硬件 IDLE 检测逻辑会捕捉到这个空闲状态，自动置位 `IDLE` 标志并**触发 IDLE 中断。**这是硬件直接产生的信号，精准且不受 CPU 负载影响。

软件层：IDLE 中断 & 主循环

IDLE 中断服务程序（ISR）——只做最少的记录工作:

1. 读取 DMA 的 `NDTR` 寄存器，计算出本次接收的字节数：`本次长度 = 缓冲区总大小 - NDTR`。
2. 将这个长度信息（或一个 " 有新数据 " 的标志）记录下来，然后**立即退出中断。绝不在 ISR 中做数据拷贝或帧解析。**

主循环——真正的解析工作在这里:

1. 检查 IDLE 中断是否报告了新数据。
2. 如果有，根据记录的 " 本次长度 " ，从 DMA 的环形缓冲区中取出这一批新字节。
3. 将这批字节 " 喂 " 给你的**帧级状态机**进行解析。

代码实现

流程梳理

1. 编写启动文件

跟我们之前走 LED 点灯的时候的程序流程一样，硬件是不认识我们的代码的，它只认中断向量表。

所以中断向量表里必须包含我们要用到的所有东西：中断函数入口、主函数入口、堆栈指针初值。

配置堆栈

Cortex-M4 内核被设计为在复位时，**自动从 Flash 的起始地址（通常是`0x08000000`）读取两个字**：第一个字作为栈顶指针（SP）的初始值，第二个字作为复位处理函数（Reset_Handler）的入口地址。

重点：中断向量表

我们要用到的中断函数就两个：`USART2_IRQHandler` 和 `USART6_IRQHandler` 。

但是手册里的“ 240个 ”是 ARM Cortex-M4 内核架构上允许的最大外部中断数，但这并不意味着芯片厂商会把这些位置全部实现。具体要填多少个，完全取决于芯片本身实现了多少中断。

对于 STM32F401RET6，答案很明确：只需要填充 101 个向量（16个内核 + 85个外设），而不是240个。

STM32F401RET6 属于 F401xE 系列，其向量表构成是固定的：

- **内核系统异常（16个）**：这是所有 Cortex-M4 芯片都一样的，从栈顶指针、复位向量到 SysTick，固定占 16 个位置。
- **芯片外部中断（85个）**：这是 ST 为 F401xE 系列实际实现的外设中断，从 IRQ0（WWDG）到 IRQ84（最后一个外设中断）。RIOT OS 等开源项目在适配 F401xE 时，也明确定义了 `CPU_IRQ_NUMOF` 为 **85**。

所以，**16 + 85 = 101**。在 mbed 等官方或半官方的开发环境中，针对 F401RE 的配置也是明确保留 **101 个向量**（`NVIC_NUM_VECTORS 101`）。

Cortex-M4 内核的 NVIC 设计上支持最多 240 个外部中断，但这只是一个“ 容量上限 ”。**硬件不会因为你没填满 240 个就去访问那些不存在的区域。**

原因在于，**中断触发的前提是“ 外设存在且被使能 ”**。F401 芯片内部物理上只制造了 85 个可以产生中断的外设源。只要你不去使能一个不存在的中断，硬件就永远不可能触发它，也就永远不会去读取向量表中 101 之后的内容。ARM 手册中画出的 0x03FC 只是内核的“ 最大寻址范围 ”，并非每个芯片的实际边界。

2. 内存划分

我们需要提前分配好存放 DMA 搬运数据过去的内存、解析完以后得到的数存放的内存等等。

3. 晶振配置

基于曾经 F103 的经验：C 语言的标准库是会默认帮你把主频拉到 72MHz 的，但是如果自己写汇编启动的话，没有自行设置就会变成默认的 8MHz 内部晶振。

所以对于我们无人机的 F401 来说，我们同样需要配置晶振（当然，配置是在主函数里实现的）。

4. 外设初始化

RCC 初始化、UART6 初始化、DMA 初始化等等，都需要进行配置。

```
    USART_InitTypeDef USART_InitStruct;

    USART_InitStruct.USART_BaudRate            = 100000;
    USART_InitStruct.USART_WordLength          = USART_WordLength_8b;
    USART_InitStruct.USART_StopBits            = USART_StopBits_2;
    USART_InitStruct.USART_Parity              = USART_Parity_Even;
    USART_InitStruct.USART_Mode                = USART_Mode_Rx | USART_Mode_Tx;
    USART_InitStruct.USART_HardwareFlowControl = USART_HardwareFlowControl_None;

    USART_Init(USART6, &USART_InitStruct);
```

```
    DMA_InitTypeDef DMA_InitStruct;

    DMA_InitStruct.DMA_Channel            = DMA_Channel_5;          // 固定通道 5
    DMA_InitStruct.DMA_PeripheralBaseAddr = (uint32_t)&USART6->DR; // 源：串口数据寄存器
    DMA_InitStruct.DMA_Memory0BaseAddr    = (uint32_t)sbus_rx_buf; // 目的：25 字节接收缓冲
    DMA_InitStruct.DMA_DIR                = DMA_DIR_PeripheralToMemory; // 外设 → 内存
    DMA_InitStruct.DMA_BufferSize         = SBUS_FRAME_LEN;         // 25 字节
    DMA_InitStruct.DMA_PeripheralInc      = DMA_PeripheralInc_Disable; // 外设地址固定
    DMA_InitStruct.DMA_MemoryInc          = DMA_MemoryInc_Enable;      // 内存地址递增
    DMA_InitStruct.DMA_PeripheralDataSize = DMA_PeripheralDataSize_Byte; // 字节宽度
    DMA_InitStruct.DMA_MemoryDataSize     = DMA_MemoryDataSize_Byte;     // 字节宽度
    DMA_InitStruct.DMA_Mode               = DMA_Mode_Normal;        // 搬完 25 字节自动停止
    DMA_InitStruct.DMA_Priority           = DMA_Priority_High;      // 高优先级，避免丢字节
    DMA_InitStruct.DMA_FIFOMode           = DMA_FIFOMode_Disable;  // 字节流，不用 FIFO
    DMA_InitStruct.DMA_FIFOThreshold      = DMA_FIFOThreshold_HalfFull;
    DMA_InitStruct.DMA_MemoryBurst        = DMA_MemoryBurst_Single;
    DMA_InitStruct.DMA_PeripheralBurst    = DMA_PeripheralBurst_Single;

    // 使能 USART6 的 DMA 接收请求（必须开启，DMA 才能从 DR 取数）
    USART_DMACmd(USART6, USART_DMAReq_Rx, ENABLE);

    DMA_Init(DMA2_Stream1, &DMA_InitStruct);
```

5. 编写中断函数

在 IDLE 中断触发后需要进入中断函数，在中断函数里进行信息的更新、记录。

6. 编写主循环

主循环里查询 IDLE 的信息更新记录，若记录有新信息则解析，没有就继续跑其他的飞控。

【补充】知识点

SRAM、RAM 和 Flash

可以先用一句话概括它们的分工：**Flash 是“仓库”，SRAM 是“工作台”。**

特性	SRAM (静态随机存取存储器)	Flash (闪存)
本质	属于 **RAM** 的一种。RAM 是“随机存取存储器”的统称，SRAM 是它的静态实现（无需刷新电路就能保持数据）。	属于 **ROM**（只读存储器）的演进，是可电擦写的非易失性存储器。
掉电后	**数据丢失。**	**数据不丢失。**
速度	**非常快**，与CPU同频工作，是CPU的“贴身工作台”。	读取速度尚可，但**写入速度远慢于SRAM**，且擦除操作很慢。
在STM32中的角色	**运行时数据区。**存放程序运行中的**变量**（全局变量、局部变量、堆栈）、**中间计算结果**、**缓冲区**等。	**程序存储区。**存放你编写的**代码指令**（`.text`）、**常量**（`.rodata`）、**中断向量表**，以及**已初始化的全局变量的初始值。**
你的SBUS场景	`rx_buf`（DMA缓冲区）、`sbus_buf`（帧组装区）、`channels`（输出区）、`rx_head`/`rx_tail` 指针，**全部必须放在SRAM中。**	你写的**汇编指令**、状态机的**跳转逻辑**、**常量**（比如帧头 `0x0F`）存放在 Flash 中。

关键澄清：在日常对话中，“RAM”通常就是指 SRAM（在 STM32 这类 MCU 里，主 RAM 就是 SRAM）。当你说“把数据放进 RAM”，在 STM32 语境下，就是指放进 SRAM。

> **一个需要关注的“空间陷阱”：栈溢出**
>
> 虽然SRAM很大，但**栈的空间通常是有限的**（默认可能只有1KB~2KB）。如果你的状态机函数里定义了**很大的局部数组**，或者在中断服务程序里做了复杂的嵌套调用，就可能耗尽栈空间，导致 **HardFault** 或数据被踩踏。
>
> **对你的建议**：在SBUS解析中，**缓冲区（`rx_buf`, `sbus_buf`, `channels`）一定要定义为全局变量（放在 `.bss` 段）**，而**不要在函数内部定义为局部数组。**这样它们就不占用宝贵的栈空间，栈只用于存放指针、索引、返回地址等小数据，安全系数会高很多。你队友在 `DATA` 段定义 `rx_buf`、`sbus_buf`、`channels` 的做法，正是遵循了这个原则。

中断向量表

硬件是按固定地址顺序来识别的，完全不关心名字。在汇编启动文件里出现 `USART2_IRQHandler` 这个名字，只是因为链接器需要它来对应正确的地址槽位。

Cortex-M4内核的NVIC（嵌套向量中断控制器）在响应一个中断（IRQ）时，完全依据**一个硬编码的“中断号（IRQ Number）”。**

这个中断号会在向量表中被转换成一个**固定的地址偏移。**例如：

- IRQ0 的中断向量地址 = `向量表基址 + 0x0040`
- IRQ1 的中断向量地址 = `向量表基址 + 0x0044`
- 以此类推，每个IRQ的偏移 = `0x40 + IRQ号 × 4`

所以，`USART2_IRQHandler` **这个名字没有任何硬件意义。**你完全可以把它改成 `MySerialHandler`，只要这个标号被放置在了 `IRQ38` 对应的那个地址槽位里，当USART2中断发生时，CPU就会跳转到那个地址去执行你的代码。

汇编启动文件里使用这些名字，只是为了**可读性**，以及方便C语言中的 `void USART2_IRQHandler(void)` 函数通过链接器覆盖掉汇编里的弱定义（`[WEAK]`）实现。

Cortex-M4 系统异常（所有Cortex-M4芯片通用，顺序固定）

IRQ号	偏移地址	异常类型	标准向量名称
—	`0x0000`	栈顶地址（非中断）	`__initial_sp`
—	`0x0004`	复位	`Reset_Handler`
-14	`0x0008`	NMI	`NMI_Handler`
-13	`0x000C`	HardFault	`HardFault_Handler`
-12	`0x0010`	MemManage	`MemManage_Handler`
-11	`0x0014`	BusFault	`BusFault_Handler`
-10	`0x0018`	UsageFault	`UsageFault_Handler`
—	`0x001C~0x002B`	保留	0（占位）
-5	`0x002C`	SVCall	`SVC_Handler`
-4	`0x0030`	DebugMon	`DebugMon_Handler`
—	`0x0034`	保留	0
-2	`0x0038`	PendSV	`PendSV_Handler`
-1	`0x003C`	SysTick	`SysTick_Handler`

这些系统异常的顺序由ARM架构固定，不可更改。

📡 STM32F401 外部中断（IRQ 0 ~ 81，部分常用列表）

从偏移 `0x0040` 开始，是STM32F401的外设中断。顺序由ST芯片设计固定，**不同型号（如F401 vs F103）顺序不同，不可跨型号混用。**

IRQ号	偏移地址	标准向量名称	说明
0	`0x0040`	`WWDG_IRQHandler`	窗口看门狗

IRQ号	偏移地址	标准向量名称	说明
1	`0x0044`	`PVD_IRQHandler`	电源电压监测
2	`0x0048`	`TAMP_STAMP_IRQHandler`	侵入/时间戳
3	`0x004C`	`RTC_WKUP_IRQHandler`	RTC唤醒
4	`0x0050`	`FLASH_IRQHandler`	Flash操作
5	`0x0054`	`RCC_IRQHandler`	时钟控制
6	`0x0058`	`EXTI0_IRQHandler`	外部中断线0
…	…	…	…
37	`0x00D4`	`USART1_IRQHandler`	串口1
38	`0x00D8`	`USART2_IRQHandler`	**串口2（你的SBUS接收）**
39	`0x00DC`	`USART3_IRQHandler`	串口3
40	`0x00E0`	`EXTI15_10_IRQHandler`	外部中断线15~10
41	`0x00E4`	`RTC_Alarm_IRQHandler`	RTC闹钟
42	`0x00E8`	`OTG_FS_WKUP_IRQHandler`	USB OTG唤醒
…	…	…	…
56	`0x0120`	`DMA1_Stream4_IRQHandler`	DMA1 Stream4
57	`0x0124`	`DMA1_Stream5_IRQHandler`	**DMA1 Stream5（USART2_RX常用）**
58	`0x0128`	`DMA1_Stream6_IRQHandler`	DMA1 Stream6
59	`0x012C`	`DMA1_Stream7_IRQHandler`	DMA1 Stream7
…	…	…	…
67	`0x014C`	`OTG_FS_IRQHandler`	USB OTG FS
68	`0x0150`	`DMA2_Stream0_IRQHandler`	DMA2 Stream0
69	`0x0154`	`DMA2_Stream1_IRQHandler`	DMA2 Stream1
70	`0x0158`	`DMA2_Stream2_IRQHandler`	DMA2 Stream2
71	`0x015C`	`DMA2_Stream3_IRQHandler`	DMA2 Stream3
72	`0x0160`	`DMA2_Stream4_IRQHandler`	DMA2 Stream4
73	`0x0164`	`DMA2_Stream5_IRQHandler`	DMA2 Stream5
74	`0x0168`	`DMA2_Stream6_IRQHandler`	DMA2 Stream6
75	`0x016C`	`DMA2_Stream7_IRQHandler`	DMA2 Stream7
76	`0x0170`	`USART6_IRQHandler`	串口6

IRQ号	偏移地址	标准向量名称	说明
77	`0x0174`	`I2C3_EV_IRQHandler`	I2C3事件
78	`0x0178`	`I2C3_ER_IRQHandler`	I2C3错误
…	…	…	…

> 完整的STM32F401向量表（含所有IRQ 0~81）可在ST参考手册 **RM0368 的 Table 38** 中查到。不同F401子型号（如F401xB/C vs F401xD/E）的向量表在个别位置可能有细微差异，**以你实际使用的型号对应的参考手册为准**。

关键校验：在你的启动文件中，`USART2_IRQHandler` 必须被放在从 `__Vectors` 算起的第 `38 + 16 = 54` 个字的位置（因为前16个位置被系统异常占用），即偏移 `0x00D8` 处。如果错放到了 `0x00D4`（USART1的位置），USART2中断就会错误地跳转到USART1的处理函数。

时钟树

1. 时钟源体系

芯片内置 4 类基础时钟源，分为高速、低速两组，可根据精度、功耗需求灵活选择：

- 高速时钟源（系统主时钟来源）
  - 内部高速 RC 振荡器（HSI）：固定 16MHz，出厂校准后 25℃下精度可达 ±1%。上电复位后默认作为 CPU 系统时钟，无需外部硬件，同时作为外部时钟故障时的备用时钟。
  - 外部高速晶振（HSE）：支持 4~26MHz 外部晶振 / 外部时钟输入，精度更高、抖动更小，是高性能场景的首选时钟源，可作为 PLL 的输入。
- 低速时钟源（低功耗与实时时钟）
  - 内部低速 RC 振荡器（LSI）：约 32kHz，主要供给独立看门狗、自动唤醒单元，无需外部元件。
  - 外部低速晶振（LSE）：标准 32.768kHz，为 RTC 实时时钟提供高精度计时基准。

2. PLL 锁相环倍频机制

HSI 或 HSE 均可输入主 PLL（锁相环）进行倍频，将低频时钟提升到芯片支持的最高主频。PLL 通过三级参数调整输出：

1. **PLLM 分频**：对输入时钟先做分频，将 PLL 输入时钟控制在 1~2MHz，降低输出抖动。
2. **PLLN 倍频**：对分频后的时钟进行倍频，得到 VCO（压控振荡器）时钟；VCO 频率必须落在**192~432MHz**的合法工作区间，这是 PLL 稳定工作的硬约束。
3. PLLP/PLLQ 输出分频
   - PLLP 分频后输出**系统时钟 SYSCLK**，最高支持 84MHz；
   - PLLQ 分频后输出 48MHz 时钟，专门供给 USB OTG FS 外设（USB 通信要求精准的 48MHz 时钟）。

以 HSI 16MHz 配置 84MHz 主频为例：`16MHz ÷ PLLM(16) × PLLN(336) ÷ PLLP(4) = 84MHz` 其中 VCO 频率为 336MHz，落在 192~432MHz 合法区间；PLLQ=7 时输出 336÷7=48MHz，满足 USB 时钟要求。

3. 总线时钟域与预分频架构

系统时钟 SYSCLK 经过多级预分频器，划分为三个核心时钟域，对应不同总线和外设，频率上限有严格硬件约束：

- **AHB 总线域（HCLK）**：SYSCLK 经 AHB 预分频器得到，最高 84MHz，供给 Cortex-M4 内核、SRAM、Flash、DMA、GPIO 端口等核心资源，是芯片的主干总线时钟。
- **APB2 高速外设域（PCLK2）**：HCLK 经 APB2 预分频器得到，最高 84MHz，挂载 TIM1、TIM9~11、USART1/6、SPI1、ADC 等高速外设。
- **APB1 低速外设域（PCLK1）**：HCLK 经 APB1 预分频器得到，**最高仅 42MHz**，挂载 TIM2~5、USART2/3、SPI2/3、I2C、窗口看门狗等低速外设。

定时器时钟补充规则：挂载在 APB2 的定时器，工作时钟最高 84MHz；挂载在 APB1 的定时器，工作时钟最高 42MHz，与对应总线的频率上限匹配。

4. 时钟安全机制（CSS）

芯片内置时钟安全系统，专门监控外部高速时钟 HSE 的工作状态：

- 若 HSE 出现故障（如晶振停振、硬件失效），硬件会**自动将系统时钟切换回内部 HSI**，避免系统死机；
- 同时可触发时钟故障中断，通知软件进行应急处理，是高可靠性场景（如飞控）的关键安全保障。

STM32F401 时钟配置要求

时钟配置必须严格遵循硬件约束和官方流程，否则会出现 Flash 访问错误、外设时序异常、甚至芯片工作不稳定的问题。

1. 核心参数约束（不可突破的硬件限制）

1. PLL 工作约束
    - PLL 分频后的输入时钟需在 1~2MHz 范围内；
    - VCO 倍频输出必须处于 192~432MHz 区间；
    - 系统主时钟 SYSCLK 最高不可超过 84MHz。
2. 总线频率上限
    - AHB 总线（HCLK）≤ 84MHz；
    - APB2 总线（PCLK2）≤ 84MHz；
    - APB1 总线（PCLK1）≤ 42MHz：当 HCLK 运行在 84MHz 时，APB1 预分频系数必须设置为 2 分频及以上，严禁超规格运行。
3. 外设时钟基准

    所有外设的波特率、通信时序计算，必须以其挂载总线的实际时钟为基准：
    - APB1 外设（如 USART2）：以 PCLK1（42MHz）计算波特率；
    - APB2 外设（如 USART6）：以 PCLK2（84MHz）计算波特率； 若错误使用系统时钟主频计算，会导致通信完全异常（如串口波特率偏差、SBUS 接收失败）。

2. 官方标准配置流程（提升主频的正确顺序）

参考手册明确要求：提升 CPU 频率时，**必须先配置 Flash 等待周期，再切换时钟源**，具体步骤如下：

1. **配置 Flash 等待周期**：根据目标主频设置 Flash 访问延迟（Latency），同时开启 Flash 预取指、指令 / 数据缓存，保证高频下 Flash 访问时序正确。
2. **预设总线分频系数**：提前将 APB1 预分频设置为 2 分频，避免时钟切换瞬间 PCLK1 超出 42MHz 上限。
3. **配置 PLL 参数**：选择 PLL 的输入时钟源（HSI 或 HSE），设置 PLLM、PLLN、PLLP、PLLQ 参数，确保所有频率参数符合硬件约束。
4. **使能 PLL 并等待锁定**：开启 PLL 后，循环等待 PLL 锁定标志位置位，确认倍频输出稳定。
5. **切换系统时钟源**：将 SYSCLK 的时钟源切换为 PLL 输出，等待时钟切换完成。
6. **校验时钟状态**：确认各总线时钟输出符合预期，完成配置。

3. 其他配置注意事项

- 外设时钟默认关闭：复位后所有外设的时钟默认处于关闭状态（降低功耗），使用任意外设前必须单独开启对应总线的外设时钟。
- 低速外设时钟独立配置：RTC、独立看门狗的时钟源需单独配置，与系统主时钟配置相互独立。